README - Tugas Praktikum P9

Pemrograman Berorientasi Obyek — Implementasi UML Class Diagram ke Program Java

Nama Mahasiswa	: Aldo Rachmad Hidayat
NRP	: 3125522003
Program Studi	: D3 Teknik Informatika - PENS PSDKU Sumenep
Project	: SiKos - Sistem Informasi Manajemen Kos (lanjutan P7)

1. Penjelasan Singkat Hubungan UML dengan Implementasi

UML Class Diagram pada halaman 2 dibuat dari source code SiKos versi P9, sehingga setiap class, attribute (private), method, dan relasi pada diagram memiliki padanan satu-satu di kode Java. Satu kotak class menjadi satu file `.java`, simbol `-` menjadi `private`, simbol `+` menjadi `public`, dan garis relasi menjadi attribute bertipe class lain atau kata kunci `extends`/`implements`.

2. Class, Responsibility, dan File

Class (UML)	File Java	Responsibility
Pengguna <<*abstract*>>	Pengguna.java	Menyimpan id dan nama; mewajibkan setiap subclass memiliki tampilkanPeran()
Penyewa	Penyewa.java	Menempati Kamar, pindahKamar(), checkOut() (hanya jika tagihan lunas), membuat Tagihan sendiri
PemilikKos	PemilikKos.java	Mengelola Kos dan menampilkan laporan kamar tersedia serta potensi pendapatan
Kamar	Kamar.java	Menjaga statusnya sendiri lewat tempati() dan kosongkan(); validasi tipe dan harga sewa
Tagihan	Tagihan.java	Memutuskan apakah pembayaran cukup lewat bayar(); status lunas hanya berubah dari dalam class
Pembayaran	Pembayaran.java	Memproses pembayaran ke Tagihan milik Penyewa lewat prosesPembayaran()
Kos	Kos.java	Menampung daftar Kamar, cariKamarKosong(), hitungPotensiPendapatan()
DapatDicekStatus <<*interface*>>	DapatDicekStatus.java	Kontrak cekStatus() yang diimplementasikan Kamar dan Tagihan

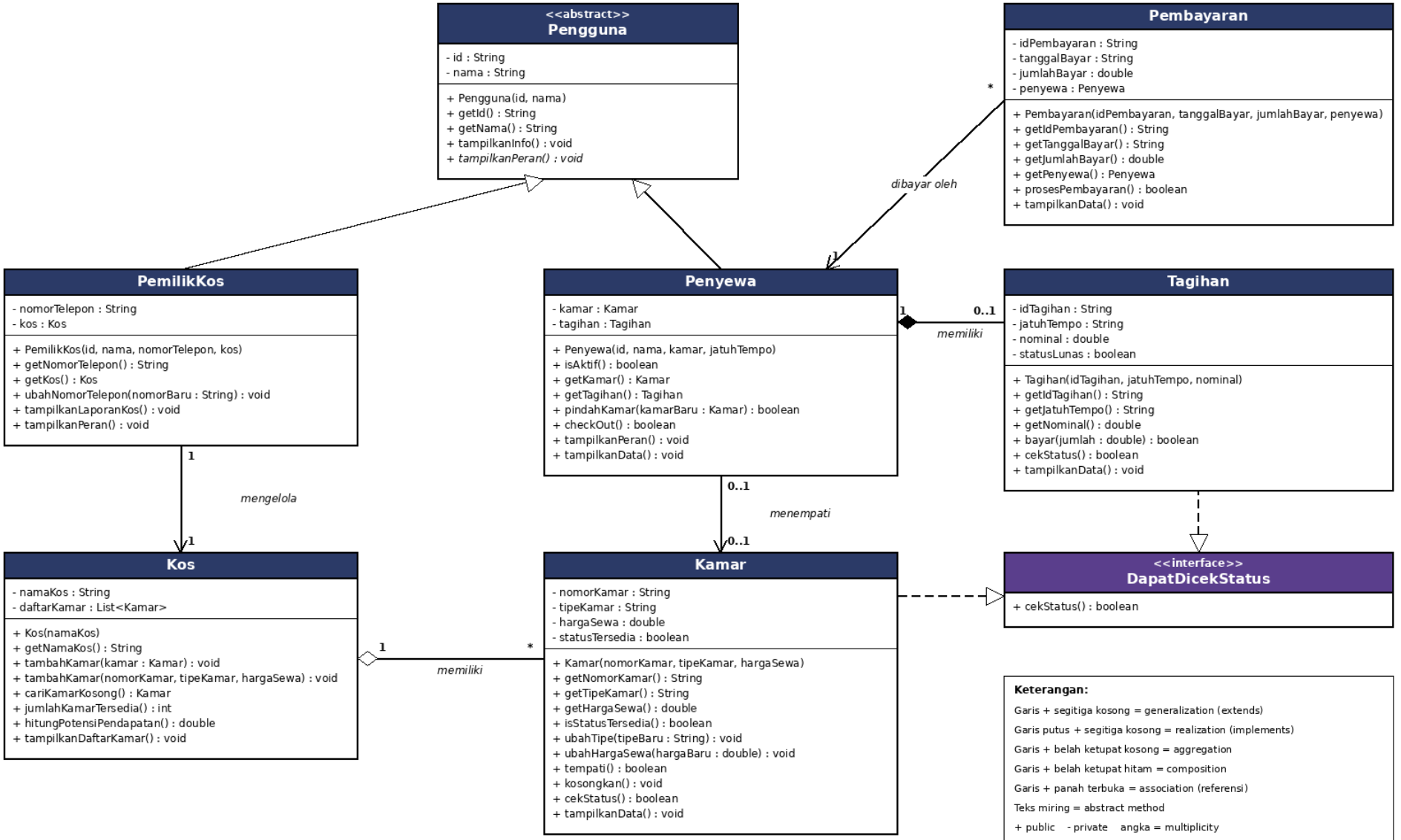
3. Relasi UML dan Implementasinya di Java

Relasi di UML	Implementasi di Java
Generalization: Penyewa, PemilikKos → Pengguna	`public class Penyewa extends Pengguna`; constructor memanggil `super(id, nama)`
Realization: Kamar, Tagihan → DapatDicekStatus	`public class Kamar implements DapatDicekStatus` beserta method `cekStatus()`
Association: Penyewa → Kamar (0..1)	`private Kamar kamar;` diisi di constructor setelah `kamar.tempati()` berhasil; bernilai null jika penyewa belum/tidak lagi menempati kamar
Association: Pembayaran → Penyewa (* ke 1)	`private Penyewa penyewa;` dikirim lewat constructor Pembayaran
Association: PemilikKos → Kos (1 ke 1)	`private Kos kos;` dikirim lewat constructor PemilikKos
Aggregation: Kos memiliki Kamar (1 ke *)	`private List<Kamar> daftarKamar;`; Kamar dapat dibuat di luar Kos (mis. `k01`) lalu dimasukkan lewat `tambahKamar(k01)`, sehingga tetap ada tanpa Kos
Composition: Penyewa memiliki Tagihan (1 ke 0..1)	`this.tagihan = new Tagihan(...)` di dalam constructor Penyewa; Tagihan tidak dibuat dari luar

4. Perubahan dari P7 agar Sesuai Ketentuan Responsibility

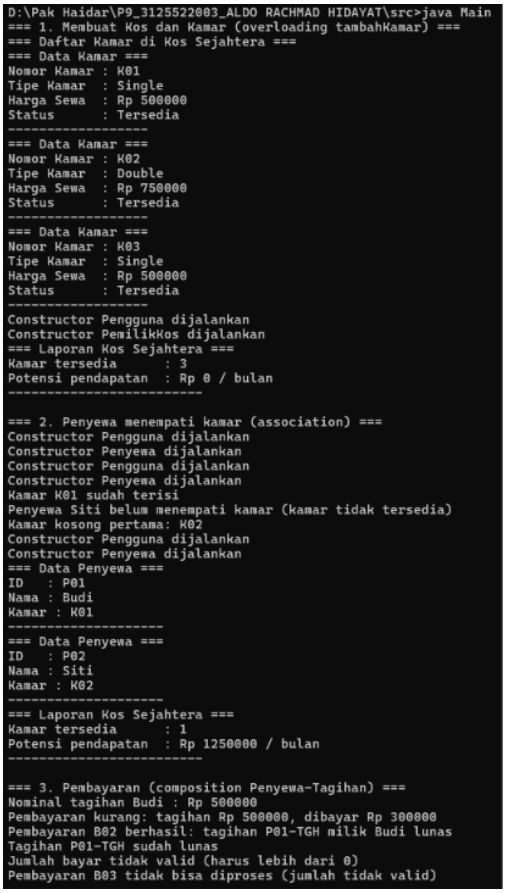
Modul P9 meminta class tidak hanya berisi attribute dan getter/setter. Karena itu setter yang mengubah state secara bebas diganti dengan method yang menjaga aturannya sendiri: `setStatusTersedia()` → `tempati()` dan `kosongkan()`; `setStatusLunas()` → `bayar()`; `setKamar()` → `pindahKamar()` dan `checkOut()`; `setJumlahBayar()` dihapus dan validasinya dipindah ke constructor Pembayaran.

5. UML Class Diagram Versi Terbaru



UML Class Diagram SiKos (P9) - Aldo Rachmad Hidayat (3125522003)

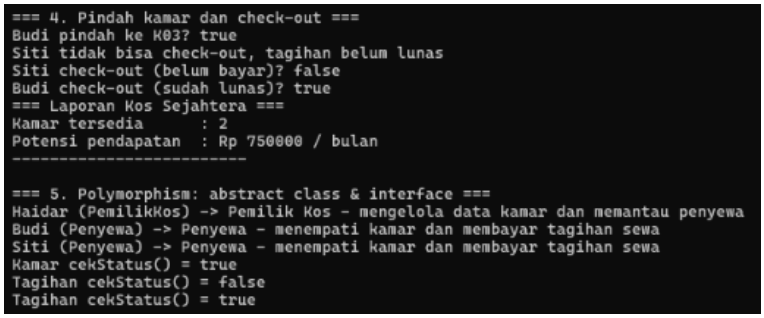
6. Screenshot Hasil Program



```
D:\Pak Haidar\P9_3125522003_ALDO RACHMAD HIDAYAT\src>java Main
=== 1. Membuat Kos dan Kamar (overloading tambahKamar) ===
=== Daftar Kamar di Kos Sejahtera ===
=== Data Kamar ===
Nomor Kamar : K01
Tipe Kamar  : Single
Harga Sewa  : Rp 500000
Status      : Tersedia
--------------------
=== Data Kamar ===
Nomor Kamar : K02
Tipe Kamar  : Double
Harga Sewa  : Rp 750000
Status      : Tersedia
--------------------
=== Data Kamar ===
Nomor Kamar : K03
Tipe Kamar  : Single
Harga Sewa  : Rp 500000
Status      : Tersedia
--------------------
Constructor Pengguna dijalankan
Constructor PemilikKos dijalankan
=== Laporan Kos Sejahtera ===
Kamar tersedia      : 3
Potensi pendapatan  : Rp 0 / bulan
-------------------------

=== 2. Penyewa menempati kamar (association) ===
Constructor Pengguna dijalankan
Constructor Penyewa dijalankan
Constructor Pengguna dijalankan
Constructor Penyewa dijalankan
Kamar K01 sudah terisi
Penyewa Siti belum menempati kamar (kamar tidak tersedia)
Kamar kosong pertama: K02
Constructor Pengguna dijalankan
Constructor Penyewa dijalankan
=== Data Penyewa ===
ID   : P01
Nama : Budi
Kamar : K01
--------------------
=== Data Penyewa ===
ID   : P02
Nama : Siti
Kamar : K02
--------------------
=== Laporan Kos Sejahtera ===
Kamar tersedia      : 1
Potensi pendapatan  : Rp 1250000 / bulan
-------------------------

=== 3. Pembayaran (composition Penyewa-Tagihan) ===
Nominal tagihan Budi : Rp 500000
Pembayaran kurang: tagihan Rp 500000, dibayar Rp 300000
Pembayaran B02 berhasil: tagihan P01-TGH milik Budi lunas
Tagihan P01-TGH sudah lunas
Jumlah bayar tidak valid (harus lebih dari 0)
Pembayaran B03 tidak bisa diproses (jumlah tidak valid)
```



```
=== 4. Pindah kamar dan check-out ===
Budi pindah ke K03? true
Siti tidak bisa check-out, tagihan belum lunas
Siti check-out (belum bayar)? false
Budi check-out (sudah lunas)? true
=== Laporan Kos Sejahtera ===
Kamar tersedia      : 2
Potensi pendapatan  : Rp 750000 / bulan
-------------------------

=== 5. Polymorphism: abstract class & interface ===
Haidar (PemilikKos) -> Pemilik Kos - mengelola data kamar dan memantau penyewa
Budi (Penyewa) -> Penyewa - menempati kamar dan membayar tagihan sewa
Siti (Penyewa) -> Penyewa - menempati kamar dan membayar tagihan sewa
Kamar cekStatus() = true
Tagihan cekStatus() = false
Tagihan cekStatus() = true
```

Skenario 1-3 (kiri atas ke bawah) membuktikan overloading, association, composition, dan validasi pembayaran. Skenario 4-5 (kanan) membuktikan pindah kamar, check-out bersyarat, serta polymorphism lewat abstract class dan interface.

7. Jawaban Pertanyaan Evaluasi

1. Apa perbedaan class pada UML dan object pada program?
Class pada UML adalah rancangan (blueprint) yang menyebutkan attribute dan method, misalnya kotak Kamar. Object adalah wujudnya saat program berjalan, misalnya `k01 = new Kamar("K01", "Single", 500000)`. Satu class dapat menghasilkan banyak object (K01, K02, K03) dengan data berbeda.

2. Bagaimana association pada UML diterapkan dalam Java?
Sebagai attribute bertipe class lain yang diisi lewat constructor atau method, contohnya `private Kamar kamar;` pada Penyewa dan `private Penyewa penyewa;` pada Pembayaran. Object yang dihubungkan tetap berdiri sendiri.

3. Mengapa attribute sebaiknya tidak seluruhnya public?
Agar state object hanya berubah lewat aturan yang dijaga class itu sendiri. Jika `statusTersedia` public, kode luar bisa mengubahnya tanpa pengecekan; dengan private, status hanya berubah lewat `tempati()` yang menolak kamar yang sudah terisi.

4. Apakah source code proyek sudah konsisten dengan UML?
Ya. Seluruh 8 class/interface, attribute private, method (termasuk constructor), dan 7 relasi pada diagram memiliki padanan di source code. Multiplicity 0..1 pada Penyewa-Kamar dan Penyewa-Tagihan sengaja dicantumkan karena di kode keduanya bisa null saat penyewa gagal mendapat kamar atau sudah check-out.

5. Class mana yang memiliki responsibility paling besar?
Penyewa, karena mengoordinasikan Kamar dan Tagihan: membuat Tagihan sendiri, menempati kamar, pindah kamar, dan check-out yang hanya boleh jika tagihan lunas. Class lain memegang tanggung jawab yang lebih sempit (Kamar menjaga status, Tagihan menilai kecukupan bayar, Pembayaran memproses).